

1. Indique para cada una de las afirmaciones si son verdaderas o falsas (Explicar cada punto)

1. El algoritmo del "banquero" no es aplicable cuando un recurso es compartido simultáneamente por varios usuarios.

VERDADERO. Este algoritmo es aplicable cuando se tienen varios recursos con varias instancias de cada uno, y varios procesos requiriendo su utilización. En otro caso, cuando hay recursos con una única instancia, se utiliza el grafo de requerimientos y asignación de recursos.

2. Un sistema multiprogramado da mejor tiempo promedio de retorno (turnaround) que un sistema no multiprogramado.

VERDADERO. La multiprogramación permite que varios procesos se encuentren listos para poder ejecutarse en el sistema. Los sistemas multiprogramados proporcionan un entorno en el que se usan de forma más eficiente los recursos del sistema, como por ejemplo la CPU, la memoria y los dispositivos de E/S.

El objetivo de la multiprogramación es tener un proceso en ejecución en todo momento para un mayor aprovechamiento de la CPU. En un sistema no multiprogramado nunca habrá más de un proceso en ejecución. Si hay más procesos, tendrán que esperar a que la CPU quede libre para reasignarse. Para un mayor aprovechamiento, se mantienen varios procesos en memoria a la vez. Cuando un proceso necesita esperar, el SO le quita la CPU y se la da a otro proceso.

De todas maneras, el tiempo de retorno dependerá exclusivamente de la política de planificación de procesos adoptada.

3. Las políticas de planificación de procesos no apropiativas son indispensables para sistemas de tiempo compartido.

FALSO. Las políticas de planificación de procesos apropiativas son indispensables para los sistemas de tiempo compartido, ya que lo que se desea es dar la sensación al usuario de que se ejecutan los procesos de manera simultánea. De no tenerse una política de planificación apropiativa, podría darse el caso de que otro proceso tome más tiempo de CPU, haciendo así esperar demasiado a los demás que se encuentren aguardando por la utilización del mismo. De este modo no se podría cumplir con el objetivo de los sistemas de tiempo compartido, que es dar la apariencia que se están ejecutando simultáneamente múltiples procesos y ofrecer a cada usuario una pequeña porción del tiempo de una computadora.

2.- Cuando no hay tareas a ejecutar, ni dispositivos de E/S que atender, ni usuarios a los cuales responder. ¿Qué hace el sistema operativo? Justifique la respuesta.

Si no hay ningún proceso o tarea que ejecutar, ningún dispositivo de E/S al que atender y ningún usuario al que responder, un sistema operativo debe permanecer inactivo, esperando a que ocurra algún evento. Los eventos siempre se indican con la ocurrencia de una interrupción o una excepción. Una excepción es una interrupción generada por software, debida a un error (división por cero, un acceso a memoria no válido) o a una solicitud específica de un programa de usuario de que se realice un servicio del sistema operativo.

1.- ¿Cuándo se invoca el planificador de largo término? En un sistema operativo de tipo microkernel, dónde lo implementaría. Justifique su respuesta.

El planificador de largo término se invoca cuando hay procesos nuevos creados en el sistema, y es el encargado de determinar cuáles son admitidos para su ejecución, es decir, cuáles podrán ser admitidos para colocarse en la cola de listos (controla el grado de multiprogramación).

2.- Dado el siguiente código

Pi

```
While (turno != i) {  
  }  
Sección Crítica  
Turno = j
```

¿Permite garantizar las propiedades de sección crítica para 2 procesos?. Justifique su respuesta.

(P0 y P1) Considere: turno está inicializado en 0. La variable i vale 0 para P0 y vale 1 para P1.

Esta implementación no permite garantizar las propiedades de la sección crítica para 2 procesos. Para que se garantizaran, faltaría agregar un arreglo bandera o flag de tipo boolean que indica si el proceso está preparado para ingresar en su sección crítica. De esa manera, "turno" indica qué proceso va a ingresar en su sección crítica, si "turno==i" indica que el proceso Pi puede ejecutar su sección crítica, y si en el arreglo "flag[i]" el valor es verdadero, quiere decir que Pi está preparado para ingresar en su sección crítica. Únicamente cuando se den ambas condiciones, el proceso podría hacer uso de su SC.

```

semaforo papel_tabaco, papel_fosforo, tabaco_fosforo, agente;

main() {

    iniciar_sem(papel_tabaco, 0, 0);
    iniciar_sem(papel_fosforo, 0, 0);
    iniciar_sem(tabaco_fosforo, 0, 0);
    iniciar_sem(agente, 0, 1);

    hilos fumador_tabaco, fumador_papel, fumador_fosforo, Agente;

    crear_hilo (fumador_tabaco, NULL, FT, NULL);
    crear_hilo (fumador_papel, NULL, FP, NULL);
    crear_hilo (fumador_fosforo, NULL, FF, NULL);
    crear_hilo (Agente, NULL, AG, NULL);

    esperar_hilo (fumador_tabaco, NULL);
    esperar_hilo (fumador_papel, NULL);
    esperar_hilo (fumador_fosforo, NULL);
    esperar_hilo (Agente, NULL);
}

void AG () {
    while(true) {
        wait(agente);
        signal(papel_tabaco);
        wait(agente);
        signal(papel_fosforo);
        wait(agente);
        signal(tabaco_fosforo);
        wait(agente); }
}

void FT () {
    while(true) {
        wait(papel_fosforo);
        print("Fumador con tabaco fumando ... \n");
        signal(agente); }
}

void FP () {
    while(true) {
        wait(tabaco_fosforo);
        print("Fumador con papel fumando ... \n");
        signal(agente); }
}

void FF () {
    while(true) {
        wait(papel_tabaco);
        print("Fumador con fosforo fumando... \n");
        signal(agente); }
}

```